CPU Scheduling

Lecture 8 — Concepts, Criteria, FCFS and SJF

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Compute Not Click

Welcome: Who Runs Next on the Lab Server?

The big question

Five students submit compile jobs to the department's lab server within five milliseconds. The server has one CPU. In what order should the jobs run — and how would you even decide that one order is better than another?

Process	P1	P2	P3	P4	P5
Arrival time (ms)	0	1	2	3	5
CPU burst τ (ms)	6	8	2	5	4

Running thread for today

We schedule these five jobs three ways — FCFS, SJF, SRTF — and measure each schedule with the same yardsticks.

Roadmap: Four Questions About the Ready Queue

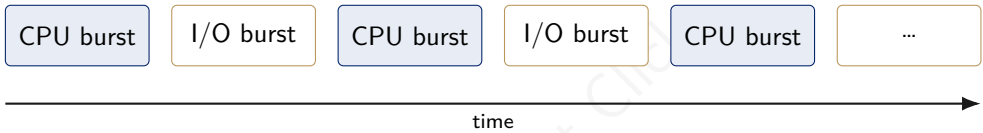
1. **Why schedule at all?** — the CPU–I/O burst cycle, the scheduler, the dispatcher, and preemption.
2. **What makes a schedule good?** — five criteria and the formulas for TAT, WT, and RT.
3. **What does the simplest policy do?** — FCFS and the convoy effect.
4. **Can we do better?** — SJF, its optimality, burst prediction, and the preemptive form SRTF.

Lecture 6 introduced the short-term scheduler as a box in the state diagram. Today we open the box and ask what policy it should run.

Concepts That Click

A process does not use the CPU continuously. It alternates between computing and waiting — and the waiting is what makes scheduling worthwhile.

The CPU–I/O Burst Cycle



- Execution is a cycle: a **CPU burst**, then an **I/O burst**, then another CPU burst, until a final CPU burst ends in termination.
- Measured burst lengths are dominated by many short bursts and a few long ones.
- An **I/O-bound** program has many short CPU bursts; a **CPU-bound** program has a few long ones.

Silberschatz et al., Ch. 5 §5.1, p. 183.

Contents that Click

The CPU Scheduler and the Ready Queue

Definition

The **CPU scheduler** (short-term scheduler) selects one process from the processes in memory that are ready to execute and allocates the CPU to it.

- The ready queue holds PCBs, not programs. It is *not* necessarily FIFO: it can be a FIFO queue, a priority queue, a tree, or an unordered list.
- The *policy* is the rule that picks from it. FCFS, SJF, and SRTF are three such rules.
- While one process waits for I/O, the scheduler gives the CPU to another — that is how multiprogramming keeps the CPU busy.

The long-term and medium-term schedulers of Lecture 6 decide *who is in memory*; today's scheduler decides *who runs*.

Concept Only

When Does the Scheduler Run? Preemptive vs. Non-preemptive

A scheduling decision may be needed when a process:

1. switches from running to waiting (an I/O request, `wait`);
2. switches from running to ready (a timer interrupt);
3. switches from waiting to ready (I/O completion);
4. terminates.

Non-preemptive (cooperative)

Decisions only in cases 1 and 4. Once a process has the CPU, it keeps it until it blocks or exits.

Preemptive

Decisions also in cases 2 and 3. The kernel may take the CPU away — and must then protect shared data from a half-finished update.

Silberschatz et al., Ch. 5 §5.1, pp. 183–187.

The Dispatcher and Dispatch Latency

Definition

The **dispatcher** is the module that gives control of the CPU to the process the scheduler selected. The time it takes to stop one process and start another is the **dispatch latency**.

- It switches context (saves one PCB, loads another — Lecture 6).
- It switches to user mode.
- It jumps to the saved PC in the new process.

Why it matters

The dispatcher runs on *every* switch. A policy that switches more often pays the dispatch latency more often — we will measure that cost on the lab-server jobs.

Socratic Check 1

Question

A process finishes its CPU burst and issues a disk `read`. Under a *non-preemptive* scheduler, must a scheduling decision be made? What about when that `read` later completes?

- Issuing the `read` is case 1 (running $\rightarrow$ waiting): the CPU is free, so the scheduler must pick someone.
- The completion is case 3 (waiting $\rightarrow$ ready): a non-preemptive scheduler lets the current process keep the CPU.
- Only a preemptive scheduler may reconsider at case 3.

Before comparing policies we need yardsticks. “Better” means nothing until we say better for whom, measured how.

Concepts Click

Five Criteria

Criterion	Meaning	Goal
CPU utilization	fraction of time the CPU does useful work	maximize
Throughput	processes completed per unit time	maximize
Turnaround time (TAT)	submission to completion, per process	minimize
Waiting time (WT)	total time spent in the ready queue	minimize
Response time (RT)	submission to the *first* time on the CPU	minimize

- The scheduling policy directly controls only **waiting time**; it does not change how long a burst computes.
- We usually optimize the *average*; interactive systems also care about the *variance* of response time.

Silberschatz et al., Ch. 5 §5.2, pp. 187–188.

The Three Per-Process Formulas

Definitions (one CPU burst per process)

For process `Pi` with arrival time AT_i, burst τ_i, first-dispatch time ST_i, and completion time CT_i:

$$\mathrm{TAT}_i = \mathrm{CT}_i - \mathrm{AT}_i, \qquad \mathrm{WT}_i = \mathrm{TAT}_i - \tau_i, \qquad \mathrm{RT}_i = \mathrm{ST}_i - \mathrm{AT}_i.$$

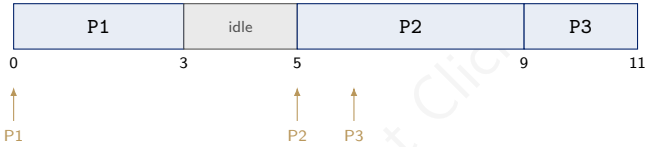
- WT counts *all* time ready but not running — including time after a preemption.
- For a non-preemptive policy, the process runs once, so $\mathrm{WT}_i = \mathrm{RT}_i$.
- Idle CPU time is not anybody's waiting time: no process is ready.

TAT, WT, and RT are spelled out here and abbreviated from now on.

Concepts

Worked Example: Metrics From a Schedule With an Idle Gap

Jobs: P1 $(0, 3)$, P2 $(5, 4)$, P3 $(6, 2)$ as (AT, τ) in ms, scheduled FCFS.



Proc.	AT	τ	CT	TAT	WT	RT
P1	0	3	3	3	0	0
P2	5	4	9	4	0	0
P3	6	2	11	5	3	3
Average				4	1	1

- The CPU idles from 3 to 5: nothing has arrived. P2 starts at its arrival, so $\mathrm{WT}_2 = 0$.
- P3: $\mathrm{TAT}_3 = 11 - 6 = 5$, $\mathrm{WT}_3 = 5 - 2 = 3$.
- Utilization over $[0, 11]$ is $9/11 \approx 81.8\%$.

Utilization and Throughput Include the Dispatcher

- Lab-server jobs: total burst $6 + 8 + 2 + 5 + 4 = 25$ ms.
- FCFS dispatches each job once: 5 dispatches.
- With a dispatch latency of 0.5 ms per dispatch, the CPU spends $25 + 5 \times 0.5 = 27.5$ ms to finish all five.

Worked numbers

$$\text{utilization} = \frac{25}{27.5} \approx 90.9\%, \qquad \text{throughput} = \frac{5 \text{ jobs}}{27.5 \text{ ms}} \approx 0.18 \text{ jobs/ms}.$$

With zero dispatch latency the same schedule gives 100% and $5/25 = 0.2$ jobs/ms. Every extra switch moves both numbers down.

Socratic Check 2

Question

For a laptop running a text editor, a browser, and a long video export, which criterion would you optimize first — and why is "maximize throughput" the wrong answer?

- The user feels **response time**: the editor must react to a keystroke within milliseconds.
- Maximizing throughput could run the export to completion first and leave the editor frozen.
- Criteria conflict; a policy is a choice of which one to favour.

The simplest rule is the one at every ticket counter: serve whoever arrived first. It is fair-sounding, easy to build — and can be very slow.

FCFS: Definition

Definition

First-Come, First-Served (FCFS) allocates the CPU to the process that requested it first. The ready queue is a FIFO queue: an arriving PCB joins the tail; the scheduler takes the head.

- FCFS is **non-preemptive**: the running process keeps the CPU until it blocks or terminates.
- The code is a linked list with a head and a tail pointer.
- Its average waiting time is often long — and depends heavily on the order of arrival.

Silberschatz et al., Ch. 5 §5.3.1, pp. 188–189; Stallings, Ch. 9 §9.2, p. 410.

Worked Example: One Long Job First

P1, P2, P3 arrive at time 0 in that order, with bursts 24, 3, 3 ms.



	P1	P2	P3
WT (ms)	0	24	27

Average WT

$(0 + 24 + 27)/3 = 17$ ms. Two 3-ms jobs wait behind one 24-ms job.

Concepts that stick

Same Jobs, Different Arrival Order

Now P2, P3, P1 arrive at time 0 in that order.



	P1	P2	P3
WT (ms)	6	0	3

Average WT

$(6+0+3)/3 = 3$ ms — a factor of $17/3 \approx 5.7$ better, with the same work and the same finish time of 30 ms.

The Convoy Effect

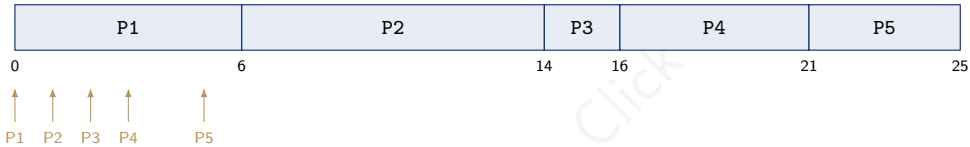
- One **CPU-bound** process holds the CPU for a long burst.
- Many **I/O-bound** processes finish their I/O and queue behind it; the I/O devices sit idle.
- The CPU-bound process finally blocks; the short jobs run quickly and block on I/O; now the CPU sits idle.
- The cycle repeats: everyone moves at the speed of the slowest.

Consequence

FCFS gives lower CPU and device utilization than letting the short jobs go first — and, being non-preemptive, it is unsuitable for time-sharing systems.

Silberschatz et al., Ch. 5 §5.3.1, pp. 188–189.

The Lab Server Under FCFS



Proc.	AT	τ	CT	TAT	WT	RT
P1	0	6	6	6	0	0
P2	1	8	14	13	5	5
P3	2	2	16	14	12	12
P4	3	5	21	18	13	13
P5	5	4	25	20	16	16
Average				14.2	9.2	9.2

- P3 needs only 2 ms but waits 12 ms behind P2's 8-ms burst. Average WT $= 46/5 = 9.2$ ms.

Concepts that Click

If order matters so much, pick the order deliberately: run the shortest job first.

Concepts Click

SJF: Definition

Definition

Shortest-Job-First (SJF) gives the CPU to the ready process with the smallest *next CPU burst*. Ties are broken FCFS.

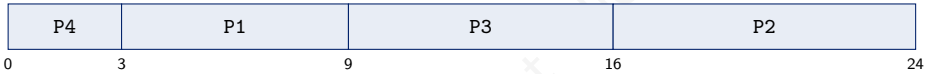
- The more accurate name is *shortest-next-CPU-burst*: the length that matters is the next burst, not the whole job.
- In the form we use first, SJF is **non-preemptive**: once chosen, a process runs its whole burst.
- The preemptive form is SRTF (later today).

Silberschatz et al., Ch. 5 §5.3.2, pp. 189–192; Stallings calls it SPN (shortest process next), Ch. 9 §9.2.

Concepts That

Worked Example: SJF With Simultaneous Arrivals

P1–P4 arrive at 0 with bursts 6, 8, 7, 3 ms.



	P1	P2	P3	P4
WT (ms)	3	16	9	0

Comparison

SJF: $(3 + 16 + 9 + 0)/4 = 7$ ms. FCFS in index order: $(0 + 6 + 14 + 21)/4 = 10.25$ ms.

Why SJF Is Optimal

- n processes all ready at time 0, run in order with bursts $\tau_{(1)}, \tau_{(2)}, \ldots, \tau_{(n)}$.
- The k-th burst delays every one of the $n - k$ processes after it:

$$\sum_i \mathrm{WT}_i = \sum_{k=1}^{n} (n-k)\,\tau_{(k)}.$$

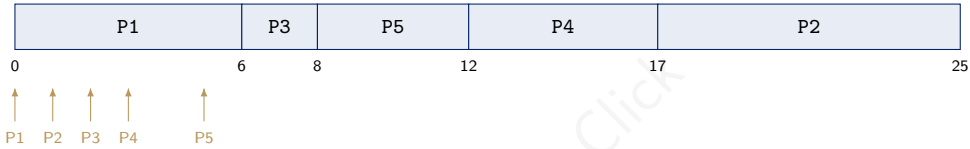
- The largest weight $n - 1$ multiplies the first burst. To minimize the sum, put the smallest burst there — and so on down.

Result

For a given set of processes, SJF gives the minimum average waiting time. Swapping any adjacent long–short pair into short–long order lowers the total.

Silberschatz et al., Ch. 5 §5.3.2, from p. 189 (SJF is provably optimal).

The Lab Server Under Non-preemptive SJF



Proc.	AT	τ	CT	TAT	WT	RT
P1	0	6	6	6	0	0
P2	1	8	25	24	16	16
P3	2	2	8	6	4	4
P4	3	5	17	14	9	9
P5	5	4	12	7	3	3
Average				11.4	6.4	6.4

- At $t = 0$ only P1 is ready, so it runs to 6. At 6 the choice is $\{8, 2, 5, 4\}$: P3, then P5, P4, P2. Average WT $= 32/5 = 6.4$ ms.

Concepts Hub Click

The Catch: Nobody Knows the Next Burst

- The short-term scheduler cannot read the future. It can only *predict* the next CPU burst from the past ones.
- The standard predictor is the **exponential average**. With τ_n the measured length of the n-th burst and $\hat{\tau}_n$ its prediction:

Exponential averaging

$$\hat{\tau}_{n+1} = \alpha\,\tau_n + (1-\alpha)\,\hat{\tau}_n, \qquad 0 \le \alpha \le 1.$$

- $\alpha = 0$: history only, recent behaviour ignored. $\alpha = 1$: only the last burst counts.
- Expanding the recursion, older bursts get weights $\alpha(1-\alpha)^j$ — each step back counts less.

Silberschatz writes t_n for the measured burst; we keep τ for bursts and add a hat for a prediction.

Worked Example: Predicting Bursts With $\alpha = 1/2$

$\hat{\tau}_0 = 10$ ms; measured bursts $6, 4, 6, 4, 13, 13, 13$ ms.

n	0	1	2	3	4	5	6	7
measured τ_n	6	4	6	4	13	13	13	—
predicted $\hat{\tau}_n$	10	8	6	6	5	9	11	12

- $\hat{\tau}_1 = \frac{1}{2}(6) + \frac{1}{2}(10) = 8$; $\hat{\tau}_2 = \frac{1}{2}(4) + \frac{1}{2}(8) = 6$; $\hat{\tau}_5 = \frac{1}{2}(13) + \frac{1}{2}(5) = 9$.
- When behaviour jumps from 4 to 13, the prediction lags — 9, 11, 12 — and converges.

Same instance as the exponential-average illustration in Silberschatz et al., Ch. 5 §5.3.2.

Concepts Matrolik

Socratic Check 3

Question

A process alternates bursts of 2 ms and 20 ms. Would you choose α close to 0 or close to 1, and what does SJF then do with it?

- α near 1 follows the last burst: it predicts 2 right before a 20 and 20 right before a 2 — always wrong.
- α near 0 settles near the long-run average (about 11 ms): never exact, but stable.
- SJF is only as good as its predictor; a bad guess turns "shortest first" into an arbitrary order.

Concepts That

Non-preemptive SJF ignores a short job that arrives one millisecond after a long one starts. The preemptive form does not.

Concepts Click

SRTF: Preemptive SJF

Definition

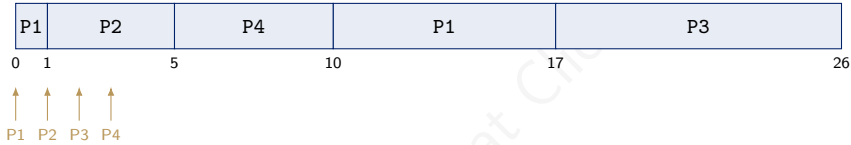
Shortest-Remaining-Time-First (SRTF) is preemptive SJF. When a process arrives, its burst is compared with the *remaining* time of the running process; if it is strictly shorter, the running process is preempted.

- Decisions happen at every arrival and every completion (cases 1, 3, and 4).
- Equal remaining times do not preempt; ties go to the earlier arrival.
- WT now includes the time a preempted process spends back in the ready queue.

Silberschatz et al., Ch. 5 §5.3.2 (from p. 189); Stallings calls it SRT, Ch. 9 §9.2.

Worked Example: SRTF vs. Non-preemptive SJF

(AT, τ): P1 $(0, 8)$, P2 $(1, 4)$, P3 $(2, 9)$, P4 $(3, 5)$.

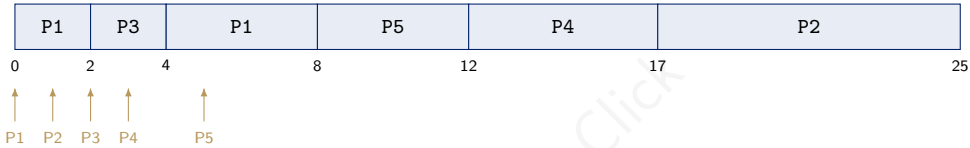


Proc.	AT	τ	CT	TAT	WT	RT
P1	0	8	17	17	9	0
P2	1	4	5	4	0	0
P3	2	9	26	24	15	15
P4	3	5	10	7	2	2
Average				13	6.5	4.25

- At 1, P2's 4 beats P1's remaining 7: preempt. P1 resumes at 10.
- SRTF average WT $= 26/4 = 6.5$ ms; non-preemptive SJF gives 7.75 ms.

Concepts that Cha

The Lab Server Under SRTF



Proc.	AT	τ	CT	TAT	WT	RT
P1	0	6	8	8	2	0
P2	1	8	25	24	16	16
P3	2	2	4	2	0	0
P4	3	5	17	14	9	9
P5	5	4	12	7	3	3
Average				11	6	5.6

- At 2, P3 (2) beats P1's remaining 4: preempt. At 3 and 5, the newcomers (5 and 4) do not beat what is running. Average WT $= 30/5 = 6.0$ ms.

Concepts in Click

Three Policies, One Workload

Policy	avg WT	avg TAT	avg RT	dispatches
FCFS	9.2	14.2	9.2	5
SJF (non-preemptive)	6.4	11.4	6.4	5
SRTF	6.0	11.0	5.6	6

- All three finish at 25 ms; the work is the same. Only the *order* — and hence the waiting — changes.
- SRTF buys the best averages with one extra dispatch.
- With 0.5-ms dispatch latency, SRTF's utilization is $25/(25 + 6 \times 0.5) = 25/28 \approx 89.3\%$, versus 90.9% for FCFS.

All times in ms; lab-server jobs from the opening slide.

Concepts That Click

The Price of Short-First: Starvation

- If short jobs keep arriving, a long job can wait *indefinitely*: SJF and SRTF can **starve** long processes.
- On the lab server, P2 (8 ms) already has the largest WT (16 ms) under both SJF and SRTF; a steady stream of 2-ms jobs would postpone it forever.
- FCFS never starves anyone: each process waits only for those ahead of it.

Two further costs

SJF/SRTF need a burst prediction, and SRTF pays a dispatch on every preemption. Ageing (raising priority with waiting time) is the usual cure for starvation — Lecture 9.

Common Misconceptions

Wrong

- "Waiting time is completion minus arrival."
- "SJF uses the total length of the job."
- "SRTF preempts on equal remaining time."
- "A better policy finishes all the work sooner."

Right

- That is TAT; $\mathrm{WT} = \mathrm{TAT} - \tau$.
- It uses the predicted *next CPU burst*.
- Only a strictly shorter newcomer preempts.
- The last completion is the same; the averages change.

Takeaways

- Processes alternate **CPU bursts** and **I/O bursts**; the short-term scheduler fills the CPU while others wait for I/O.
- Scheduling happens on block, exit (non-preemptive) and also on timer or I/O completion (**preemptive**); the **dispatcher** pays a latency on every switch.
- $\mathrm{TAT} = \mathrm{CT} - \mathrm{AT}$, $\mathrm{WT} = \mathrm{TAT} - \tau$, $\mathrm{RT} = \mathrm{ST} - \mathrm{AT}$.
- **FCFS** is simple and starvation-free but suffers the convoy effect.
- **SJF** minimizes average WT but needs a predicted burst (exponential average); **SRTF** is its preemptive form. Both can starve long jobs.

Concepts That Click

Next: Priority, Round Robin, and Multilevel Queues

- Today: what scheduling is, how to measure it, and the two policies that order by arrival and by burst length.
- Next lecture: **Priority** scheduling and ageing, **Round Robin** with a time quantum, multilevel queues and feedback queues, and scheduling on more than one CPU.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 5 §5.3–5.5 [1]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 9 §9.2 and Ch. 10 §10.1 [2].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 5 §5.1 (basic concepts, p. 183), §5.2 (scheduling criteria, p. 187), §5.3.1 (FCFS, p. 188), §5.3.2 (SJF and SRTF, p. 189).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 9 §9.1 (types of scheduling, p. 406), §9.2 (scheduling algorithms: FCFS, SPN, SRT, p. 410).

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Concepts That Click

References I


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click